ENGINEERING ARCHITECTURE & SYSTEM AUDIT REPORT

Technical Architecture and Operational Analysis of the Semester Library Chatbot Ecosystem

End-to-End Inspection of Kyana AI 3.0 RAG Engine, Multi-Provider LLM Orchestration, Cultural Persona System, RBAC Security, Web & Mobile Clients, and Peer Messaging Subsystems

Platform: Semester Library (Gandaki University BIT) | Repository: /Users/sauu_gat/semester-library | Date: Nov 3, 2026

1. Executive Overview and System Taxonomy

The conversational infrastructure within the Semester Library platform is engineered as a multi-layered ecosystem serving Bachelor of Information Technology (BIT) students, class representatives (CRs), faculty members, and administrators at Gandaki University. Rather than relying on a single monolithic chat script, the repository partitions conversational capabilities into three strictly isolated domains: the Kyana AI Study Companion (an autonomous, retrieval-augmented academic assistant), the Cohort Group Chat System (semester-bound peer and faculty rooms), and the Direct Messaging (DM) System (one-to-one private messaging).

At the center of the artificial intelligence layer is Kyana AI , which operates across both a deterministic tool-dispatch pipeline and the unified Kyana AI 3.0 Knowledge Engine . Kyana AI is architected to solve two fundamental engineering challenges inherent to university platforms operating in Nepal:

1. **Cross-Region Serverless and Database Latency:**
   Because the application's serverless compute (vercel.json) and managed PostgreSQL 16 database (db.js on Neon Serverless US East us-east-1) reside in North America, each sequential network round-trip from Kathmandu or Pokhara incurs approximately 900ms to 950ms of network latency, as documented in the Kyana 3.0 Architecture Audit . The chatbot architecture minimizes round-trips via single-query hybrid retrieval, zero-model-round-trip deterministic lookups for timetables and syllabi, and asynchronous message persistence.
2. **Linguistic and Cultural Code-Switching:**
   Students communicate across four distinct linguistic modes - English, Romanized Nepali with informal phonetic variations ("xa" , "xaina" , "khojdeu" , "bujhaideu"), mixed Nepali-English code-switching, and Devanagari script - while referencing local temporal expressions grounded in Nepal Standard Time (UTC+05:45) and the Bikram Sambat (BS) calendar via lib/nepali-dates.js .

Subsystem Domain	Primary Orchestrator Modules	Database Persistence Tables	Transport Protocol	Access & Isolation Boundary
Kyana AI Assistant	ai-assistant.js , lib/chat-http.js , lib/ai-chat-service.js	ai_conversations , ai_messages , knowledge_resources , knowledge_chunks , knowledge_jobs , note_search_documents	HTTP POST with Server-Sent Events (text/event-stream) & JSON fallback	Server-resolved academicContext enforcing student cohort and registered semester (1 - 8)
Cohort Group Chat	lib/cohort-chat.js , routes/cohort-chat.js	cohorts , chat_groups , chat_group_slots , chat_messages , chat_reactions , chat_room_read_receipts , chat_realtime_outbox	REST mutations + Supabase Realtime broadcast channels (public:chat_messages)	Active BIT cohort membership or assigned faculty teaching semesters
Direct Messaging (DM)	lib/dm-service.js , routes/direct-messaging.js	dm_conversations , dm_participants , dm_messages , dm_blocks , dm_rate_limits , dm_realtime_outbox	REST mutations + transactional outbox drained to Supabase Realtime & Push	Verified account status, participant pair enrollment, and bilateral block-list checks

2. End-to-End Request Lifecycle and Transport Layer

2.1 HTTP and Server-Sent Events (SSE) Entry Point

All client interactions with the AI chatbot converge on the POST /api/ai/chat route mounted in server.js and handled by createChatHandler(db, assistant) in lib/chat-http.js . The transport layer supports both real-time token streaming via Server-Sent Events (text/event-stream) and standard JSON responses.

When a request arrives at lib/chat-http.js , the handler executes a strict seven-stage pipeline:

1. **Payload Validation:**
   Validates that req.body.message is a non-empty string between 1 and 8,000 characters. Requests outside this boundary immediately receive an HTTP 400 Bad Request .
2. **Identity Resolution and Untrusted History Filtering:**
   Extracts studentId from the authenticated request (req.user?.studentId || req.student?.studentId || req.session?.studentId). A critical security boundary is enforced here: client-supplied assistant turns and resource cards are never trusted for authenticated users. If studentId is present, client-provided history is discarded in favor of authoritative database history; for unauthenticated guests, only prior user turns (up to the last 8 messages, truncated to 6,000 characters each) are retained.
3. **Conversation Resolution and Non-Blocking User Message Persistence:**
   For authenticated students, lib/chat-http.js verifies ownership of the requested conversationId against ai_conversations via lib/ai-chat-service.js , or provisions a default conversation (kyana_${studentId}_default). It loads the 12 most recent messages from ai_messages to reconstruct multi-turn context. Crucially, saving the incoming user message (aiChatService.saveUserMessage) is dispatched asynchronously - and registered with @vercel/functions waitUntil when available - so the database INSERT does not delay Time to First Token (TTFT).
4. **Session Mode Persistence:**
   Evaluates whether the user is toggling between roast (playful senior persona) and formal mode via resolveChatMode in lib/chat-mode.js . If req.session.chatMode changes, the session is saved to the database before SSE headers are flushed so that first-time guests receive a persistent session cookie.
5. **SSE Stream Initialization and Heartbeat Management:**
   If req.body.stream === true or the Accept header includes text/event-stream , the handler writes HTTP 200 headers (Content-Type: text/event-stream; charset=utf-8 , Cache-Control: no-cache, no-transform , X-Accel-Buffering: no), flushes headers immediately, and starts a 15-second : keepalive\n\n heartbeat timer to prevent reverse proxies or serverless gateways from dropping idle connections during retrieval. An AbortController is wired to req.on('aborted') and res.on('close') to cancel downstream database queries and LLM streams immediately if the user disconnects or taps the stop button.
6. **Core Assistant Execution:**
   Invokes assistant.handleChat(...) in ai-assistant.js , passing an onDelta callback that emits event: delta\ndata: {"text":"..."}\n\n frames as tokens arrive.
7. **Assistant Turn Persistence and Terminal Frame:**
   Once handleChat resolves, saveUserPromise is awaited to guarantee chronological ordering in ai_messages , followed by aiChatService.saveAssistantMessage(db, studentId, conversationId, result.reply, result) , which stores the reply alongside a JSON metadata snapshot containing resourceCards , evidenceReferences , and a cryptographic accessContext hash. Finally, the server emits event: result\ndata: {...}\n\n and closes the stream.

2.2 Companion AI Routes

Beyond the primary chat stream, server.js exposes eight supporting routes for conversation management, inbox previews, and live resource revalidation:

HTTP Method & Path	Handler Module	Authentication	Functional Responsibility
POST /api/ai/chat	lib/chat-http.js	Optional (Guest or Auth)	Primary streaming SSE and JSON chat execution pipeline
GET /api/ai/preview	lib/ai-chat-service.js	Optional	Returns pinned inbox preview (title , subtitle , timestamp) for unified chat lists
GET /api/ai/resources/:type/:id/open	lib/kyana-resource-http.js	requireLogin	Revalidates user permission against live source tables before redirecting or returning JSON card metadata
GET /api/ai/conversations	lib/ai-chat-service.js	requireLogin	Lists all ai_conversations belonging to the authenticated student ordered by updated_at DESC

HTTP Method & Path	Handler Module	Authentication	Functional Responsibility
GET /api/ai/conversations/:id/messages	lib/ai-chat-service.js	requireLogin	Fetches up to 100 messages for a conversation and revalidates every historical assistant message's accessContext and resourceCards
POST /api/ai/conversations	lib/ai-chat-service.js	requireLogin	Creates a new conversation record (chat_${Date.now()}_${random})
POST /api/ai/conversations/:id/clear	lib/ai-chat-service.js	requireLogin	Deletes all rows in ai_messages for the conversation while preserving the ai_conversations container
DELETE /api/ai/conversations/:id	lib/ai-chat-service.js	requireLogin	Permanently deletes both ai_messages and the parent ai_conversations row
GET /api/ai/suggestions	server.js	Public	Returns starter prompt chips for empty chat states

3. Core Orchestration Engine and Intent Routing

3.1 Context Window Compaction and Academic Context Resolution

Inside handleChat in ai-assistant.js , incoming messages are trimmed to 8,000 characters and passed through resolveChatMode (lib/chat-mode.js) and sensitiveRequest (lib/chat-security.js). To prevent context window bloat on long conversations while preserving topical continuity, handleChat compacts conversation histories exceeding 12 messages: it extracts the first 40 characters of up to 3 distinct earlier user turns, synthesizes a [Earlier context summary: student previously discussed ...] prefix message, and retains the 8 most recent turns in full (up to 6,000 characters per turn).

Next, handleChat resolves the user's authoritative academic permissions via getAcademicContext(db, studentInfo) in lib/academic-context.js . This server-side lookup queries students , cohorts , and teacher_subjects to construct an immutable permission object containing the user's role (student , cr , teacher , or admin), permanent cohort.id , dynamic cohort.currentSemester (1 - 8), and assignedSemesters (for teachers). A SHA-256 digest of this permission state is computed via contextKey(academicContext) in lib/chat-security.js and attached to every response and cache entry.

3.2 Deterministic NLU and Multi-Turn Intent Router

Before invoking an LLM or querying the database, lib/chat-tools.js executes routeQuery(message, cleanHistory) , a deterministic Natural Language Understanding (NLU) classifier that parses English, Romanized Nepali, and Devanagari queries into structured routing intents:

- **Subject and Semester Entity Extraction (extractFilters):**
  Maps Roman numerals (I - VIII), English ordinals (first - eighth , 1st - 8th), and numeric strings (sem 3 , semester 2) to canonical integer semesters 1 - 8 . Matches 26 BIT courses from public/syllabus-data.json against a curated alias dictionary (SUBJECT_ALIASES in lib/chat-tools.js , such as 'dbms' -> 'Database Management System' , 'dsa' -> 'Data Structure and Algorithms' , 'coa' -> 'Microprocessor and Computer Architecture'), sorting by longest alias match so "Mathematics II" never falsely triggers "Mathematics I" . Also infers implicit subjects from 29 topic clusters in lib/chat-topics.js (e.g., mapping "1nf" , "bcnf" , or "functional dependency" to Database Management System in Semester 3, or "k-map" and "flip flop" to Digital Logic in Semester 2).
- **Nepali Temporal Expression Resolution (parseNepaliDateExpression):**
  Invokes lib/nepali-dates.js to convert natural Nepali time words into Asia/Kathmandu (UTC+05:45) ISO date strings (YYYY-MM-DD) and date ranges: aaja / aile (today), voli / bholi (tomorrow), parsi (day after tomorrow), hijo (yesterday), asti (2 days ago), yo hapta (this week), arko hapta (next week), and gako hapta (last week).
- **Short Semester Follow-Ups:**
  If a user asks "What's in the syllabus?" and follows up with "and sem 3?" or "what about 2nd semester?" , routeQuery inherits the prior intent (notes , syllabus , or routine) and updates filters.semester .
- **Query Refinement (matra / only):**
  Short follow-ups containing "matra" or "only" (up to 4 words, such as "CSS ko matra") inherit the prior search category (notes , posts , notices , assignments) and narrow the subject/topic filter.
- **Pagination (arko xa? / more):**
  Follow-ups like "arko xa?" , "aru dekhauna" , or "any more?" re-dispatch the previous search with isMore: true .
- **Relative Reference Resolution (relative_reference):**
  Phrases combining ordinal or deictic markers ("first" , "second" , "2nd" , "pahilo" , "dosro" , "tesro" , "tyo" , "arko")

with open actions ("khola" , "open" , "view" , "dekhauna" , "heram") route to kind: 'relative_reference' with a zero-based index . In ai-assistant.js , this resolves against the previous turn's resourceCards (from history, ai_messages metadata via getLastCardsForConversation , or the in-memory lastCardsPerSession map), revalidates permissions via revalidateCards , and returns the targeted card with "${targetCard.title} open gardai chu." in zero LLM calls.

3.3 Execution Path Bifurcation: Kyana v3 Knowledge Path vs. Legacy Path

Once routeQuery determines the intent, ai-assistant.js checks isKnowledgeV3Enabled(studentId) from lib/knowledge-schema.js :

1. **Path A - Kyana AI 3.0 Unified Knowledge Retrieval (v3Enabled && isKnowledgeQuery):**
   Triggered when KYANA_KNOWLEDGE_V3 === 'true' and the student is covered by KYANA_KNOWLEDGE_V3_USERS ('*' or comma-separated student ID allowlist), for all queries touching posts , notices , assignments , notes , university policy, or academic explanations. Executes a single hybrid vector + Full-Text Search (FTS) query against Neon PostgreSQL (searchKnowledge in lib/knowledge-search.js), resolves deictic follow-ups ("yo notice ko meaning bujhaideu"), wraps retrieved records in <untrusted_campus_record> tags, performs one single streaming LLM call with tools: [] , extracts <referenced_records> IDs, and revalidates the referenced cards against live database permissions.
2. **Path B - Deterministic Zero-LLM Lookups (directTool):**
   When v3Enabled is active, get_syllabus , get_routine , and web_search remain on the direct tool execution path because syllabus JSON and exam timetable tables are deterministic. When v3Enabled is off, search_notes , search_notices , search_posts , and search_assignments also use directTool to format results via formatToolResult and styleLookupReply (lib/chat-personality.js) with zero LLM round-trips.
3. **Path C - General Academic, Code, and Multi-Tool Agent Loop:**
   For route.kind === 'direct' (when v3Enabled is off) or route.kind === 'tools' , ai-assistant.js pre-fetches up to 2 matching study note passages from lib/note-search.js if the message is an academic query, injects them into noteEvidencePrompt , and calls generateReply in lib/chat-provider.js .

4. Multi-Provider LLM Engine and Tool-Calling Protocol

4.1 Provider Priority Chain and Model Selection

The LLM communication layer in lib/chat-provider.js is implemented using native Node 18+ fetch and Web Streams without heavy third-party SDK dependencies. It supports five upstream AI providers arranged in an automatic failover chain: Gemini -> Groq -> OpenRouter -> OpenAI -> DeepSeek .

Priority Order	Provider Identifier	Endpoint Contract	Default Standard Model (complex: false)	Default Complex Model (complex: true)	Intra-Provider Candidate Fallbacks
1 (Primary)	gemini	Google AI REST streamGenerateContent?alt=sse	GEMINI_MODEL or gemini-3-flash-preview	GEMINI_COMPLEX_MODEL or gemini-3-flash-preview	gemini-3-flash-preview -> gemini-3.1-flash-lite-preview -> gemini-flash-latest (on HTTP 404/429/503)
2	groq	OpenAI-compatible /openai/v1/chat/completions	GROQ_MODEL or llama-3.3-70b-versatile	GROQ_COMPLEX_MODEL or llama-3.3-70b-versatile	llama-3.3-70b-versatile -> qwen/qwen3.8-27b -> openai/gpt-oss-120b (on HTTP 404)
3	openrouter	OpenAI-compatible /api/v1/chat/completions	OPENROUTER_MODEL or google/gemini-3.1-flash-lite	OPENROUTER_COMPLEX_MODEL or google/gemini-3.1-pro-preview	Configured via OpenRouter routing
4	openai	OpenAI REST /v1/chat/completions	OPENAI_MODEL or gpt-4o-mini	OPENAI_COMPLEX_MODEL or gpt-4o	None
5	deepseek	OpenAI-compatible /chat/completions	DEEPSEEK_MODEL or deepseek-chat	DEEPSEEK_MODEL or deepseek-chat	None

Generation parameters are standardized at temperature: 0.25 and maxOutputTokens: 3000 (5000 when complex: true). For Gemini 2.5 Flash models, thinkingConfig: { thinkingBudget: 0 } is set; for Gemini 3 Flash models, thinkingConfig: { thinkingLevel: 'minimal' } is passed to minimize time-to-first-token latency.

4.2 Bounded Two-Round Tool Execution and Speculative Prose Buffering

When tools are provided to generateReply in lib/chat-provider.js , the provider runs a strictly bounded loop of at most 2 rounds and at most 4 parallel tool calls (MAX_TOOL_CALLS = 4):

- **Round 0 (Tool Selection & Execution):**
  Tools are enabled (allowTools = true). Because some LLMs emit speculative prose before emitting a function call, onDelta chunks during Round 0 are buffered in pendingChunks rather than streamed immediately to the client. If the model finishes Round 0 without calling any tools, the buffered chunks are flushed to the user.
- **Strict Argument Validation (argumentsFor):**
  Each tool call is validated against the JSON Schema in TOOLS before execution: prototype pollution keys (__proto__ , prototype , constructor) are rejected, unknown properties throw INVALID_TOOL_CALL , string lengths are capped at maxLength (default 2,000 characters, total JSON payload <= 16,000 bytes), and numeric bounds (minimum: 1, maximum: 8 for semesters) are enforced.
- **Deduplicated Parallel Execution:**
  Tool calls are deduplicated by JSON.stringify([call.name, call.args]) and executed concurrently via Promise.all , wrapped in withDeadline (default 15,000ms timeout, configurable via CHAT_TIMEOUT_MS between 1,000ms and 30,000ms). Tool outputs exceeding 64,000 serialized bytes are replaced with { error: 'RESULT_TOO_LARGE', message: 'Narrow the search by subject or semester.' } .
- **Round 1 (Final Synthesis):**
  Tool outputs are appended to contents (preserving exact thought signatures for Gemini functionResponse turns) or messages , tools are disabled (allowTools = false), and the final synthesis tokens are streamed directly to the client.
- **Clean Failover Guarantee:**
  If a provider fails before any visible text is emitted to the user (!visibleText), generateReply catches the error and transparently falls back to the next configured provider in defaultChain . Once any token has been streamed to the client, failover is prohibited to prevent concatenating two partial answers.

5. Kyana AI 3.0 Unified Knowledge Engine (RAG Pipeline)

5.1 Unified PostgreSQL Schema (pgvector + Weighted Multilingual FTS)

Defined in migrations/20261014_kyana_v3_knowledge.up.sql and managed by lib/knowledge-schema.js , the Kyana 3.0 RAG engine consolidates eight campus resource types (post , notice , assignment , note , routine , syllabus , gu_official , comment) into three tables in Neon PostgreSQL 16:

- **knowledge_resources :**
  Stores canonical metadata (id formatted as ${type}:${canonicalId} , title , source_authority_tier 1 - 5), denormalized scoping columns (author_id , author_role , cohort_id , semester_no , audience_scope , target_all_semesters , visibility), pre-computed client routes (web_url , mobile_route , mobile_params), lifecycle timestamps (created_at , updated_at , deleted_at , hidden_at), and a deterministic SHA-256 content_hash .
- **knowledge_chunks :**
  Stores segmented text passages (id formatted as ${resource_id}:c${chunk_index}), optional PDF/slide page_number , 768-dimensional embeddings (embedding vector(768) indexed via USING hnsw (embedding vector_cosine_ops)), and a generated stored tsvector column (tsv_content) combining English stemming (setweight(to_tsvector('english', content), 'A')) with dictionary-agnostic token matching for Romanized Nepali and code (setweight(to_tsvector('simple', content), 'B')), indexed via USING GIN (tsv_content) .
- **knowledge_jobs :**
  A persistent, serverless-safe retry queue tracking asynchronous indexing operations (index , update , delete , sync) with revision counters, attempts / max_attempts (5), locked_at timestamps, and a partial unique index uq_kj_pending_resource_op preventing duplicate pending jobs for the same resource.

5.2 Document Ingestion, Chunking, and Embedding Pipeline

When campus content is created or updated, lib/knowledge-builder.js transforms raw database rows into structured { resource, chunks } payloads:

- **Contextual Chunking (chunkText):**
  Text bodies are normalized and split into chunks targeting 750 characters (600 - 800 range) with a 100-character sliding overlap , breaking cleanly on paragraph newlines, sentence periods, or word spaces past the 70% mark. Every chunk is prepended with a bracketed metadata header - such as [Type: Notice | Title: ... | Subject: ... | Audience: ...] - so that vector

embeddings capture document identity on every passage.

- **Page-Aware Study Material Extraction (lib/note-search.js):**
  Uploaded files (up to MAX_FILE_BYTES = 20 MB and MAX_TEXT_CHARS = 1,000,000) are fetched from Supabase Storage (library_files bucket), local /uploads (/tmp/uploads on Vercel), or the file_blobs table. PDFs are parsed via pdfjs-dist/legacy/build/pdf.mjs up to 500 pages, joining pages with form-feed (\f) delimiters so buildNoteKnowledge preserves exact 1-indexed pageNumber values per chunk. DOCX and PPTX files are validated against zip-bomb attacks via central-directory inspection (checkDocxSize , capping expanded size at 60 MB) before extracting text with mammoth or node-pptx-parser .
- **Vector Embeddings (lib/knowledge-embeddings.js):**
  Calls Google's gemini-embedding-001 model (embedContent for single queries with taskType: 'RETRIEVAL_QUERY' , and batchEmbedContents in batches of up to 100 chunks with taskType: 'RETRIEVAL_DOCUMENT'), requesting outputDimensionality: 768 and L2-normalizing every returned vector.

5.3 Real-Time Sync Hooks, Background Worker, and Gandaki University Sync

To keep the knowledge index synchronized with live campus activity without slowing down user mutations, lib/knowledge-sync.js and lib/knowledge-worker.js implement a dual-mode synchronization strategy:

- **Immediate Non-Blocking Upserts:**
  When a resource is created or edited, persistKnowledgeResource compares the new SHA-256 content_hash against knowledge_resources.content_hash . If unchanged, it updates metadata columns in place without re-embedding. If changed, it attempts to batch-embed and transactionally replace chunks within HOOK_TIMEOUT_MS (8,000ms). If embedding fails or times out, it indexes the text chunks with embedding = NULL (so keyword FTS works immediately) and enqueues a retry job in knowledge_jobs .
- **Synchronous Revocation on Deletion or Moderation:**
  Deletions and content moderation hides execute synchronously before the HTTP response returns, setting deleted_at = CURRENT_TIMESTAMP or hidden_at = CURRENT_TIMESTAMP on both the resource and any child comment records.
- **Internal Worker & Gandaki University Portal Sync (routes/internal-knowledge.js):**
  Protected by constant-time crypto.timingSafeEqual verification of INTERNAL_CRON_SECRET , four endpoints maintain index health: /api/internal/knowledge/worker (drains pending jobs using FOR UPDATE SKIP LOCKED), /api/internal/knowledge/reconcile (detects missing or orphaned records and hash drift), /api/internal/knowledge/sync-university (polls Gandaki University's WordPress REST API via lib/university-sync.js and indexes official notices at Tier 1), and /api/internal/knowledge/timings (exposes the 50-request latency ring buffer from lib/knowledge-timing.js).

5.4 Single-Query Hybrid Retrieval and Relevance Reranking

When searchKnowledge(db, academicContext, query, options) in lib/knowledge-search.js executes against PostgreSQL, it performs three stages:

1. **Query Tokenization, Synonym Expansion, and Embedding:**
   lib/knowledge-relevance.js strips English, Romanized Nepali, and Devanagari stopwords and expands terms through ALIASES (mapping Devanagari terms like -> exam , -> notice , -> routine , -> assignment , and common student typos like normalisation -> normalization , pythn -> python). Concurrently, embedText computes the 768-dim query vector with a 2,000ms timeout.
2. **In-SQL Live Table Authorization Join and Passage Diversification:**
   Executes a single SQL CTE (WITH candidates AS (...)) that joins knowledge_resources and knowledge_chunks directly against the live source tables (posts , assignments , files , note_search_documents , routine , post_comments) and injects the role-and-semester SQL predicates generated by resourceFilter (lib/kyana-access.js). A window function (ROW_NUMBER() OVER (PARTITION BY id ORDER BY fts_score DESC, vector_sim DESC)) caps passages at 3 chunks per resource.
3. **Live Freshness Verification and Hybrid Reranking (isFresh & rankCandidates):**
   Every returned candidate row is verified against its live source JSON via isFresh(row, row.source) , recomputing the canonical content_hash on the fly. Next, validateRelevance enforces a Named Entity Gate (extractNamedEntities) to eliminate false-positive semantic matches on out-of-domain proper nouns, and rankCandidates computes the composite score (requiring score >= 0.35): Score = ExactTitleBonus(1.0) + (0.65 x TermCoverage) + (0.20 x TitleCoverage) + (0.10 x VectorSim) + (0.05 x (6 - AuthorityTier) / 5)

Source Authority Tier	Assigned Resource Categories	Prompt Attribution Label	Conflict Resolution Weight

Source Authority Tier	Assigned Resource Categories	Prompt Attribution Label	Conflict Resolution Weight
Tier 1	Gandaki University Official Portal (gu_official), Admin Notices, Admin Comments, Exam Routines	Official College Notice	Highest institutional authority; overrides informal posts
Tier 2	Teacher Notices, Teacher Assignments (assignments), Teacher Comments	Teacher Assignment / Teacher Notice	Authoritative for course-level deadlines and clarifications
Tier 3	Class Representative (CR) Announcements, CR Comments, Teacher/CR Uploaded Notes	Class Representative (CR) Announcement / Course Study Material (File)	Cohort-level coordination and official lecture slides
Tier 4	Student Community Posts (posts), Student Comments, Student Uploaded Notes	Student Community Post / Course Study Material (File)	Peer discussion; explicitly never treated as university policy

6. Persona Engineering, Language Adaptation, and Security Architecture

6.1 Cultural Persona and Four-Way Language Adaptation

All persona rules are centralized in lib/chat-personality.js (PERSONALITY_VERSION = '2.2.0'), which serves as the single source of truth across Web and Mobile. Kyana is engineered to sound like a sharp, supportive Nepali university senior rather than a corporate chatbot:

- **Anti-Robotic Guardrails:**
  Strictly forbids canned openings ("Certainly!" , "Great question!" , "As an AI assistant..."), bans unsolicited closing questions ("Bujheu?" , "Kehi help chaiyo?"), prevents repeating the user's prompt verbatim, and forbids fabricating human experiences.
- **Four-Way Language Detection (detectLanguage & detectExplicitLanguageSwitch):**
  Automatically detects Devanagari Nepali (Unicode range [\u0900-\u097F]), Romanized Nepali (matched against NEPALI_MARKERS such as xa , xaina , hunxa , vayo , garxu , bujhaideu , khojdeu , voli , yrr , huss , dami , babal , jhyau), Mixed Nepali-English Code-Switching , and English , while honoring explicit switch commands ("English ma explain gara" , "Feri Nepali ma vana").
- **Dynamic Session Modes (toneInstructions):**
  Switches between Empathetic Support Mode (triggered automatically when stressed(message) detects exam anxiety or panic keywords like "padhei sakiyena" , "tension" , "voli exam" , or), Formal / Restrained Mode (toggled via lib/chat-mode.js), and Roast / Friendly & Witty Mode (default).

6.2 Multi-Layered Security, Access Control, and Prompt Injection Defense

Security across the chatbot system is enforced through six independent layers in lib/chat-security.js , lib/kyana-access.js , and lib/chat-ratelimit.js :

1. **Pre-Flight Prompt Injection & Secret Exfiltration Filter (sensitiveRequest):**
   Normalizes input using Unicode NFKC and blocks any request attempting to extract system prompts, developer instructions, environment variables, DATABASE_URL , API keys, session tokens, admin credentials, student password hashes, or unpublished exam papers, as well as instruction-override patterns ("ignore/disregard/bypass previous/system instructions").
2. **Untrusted Data Encapsulation (<untrusted_campus_record>):**
   Every database passage injected into the LLM prompt has XML boundary tags stripped/escaped from its title and body and is wrapped in an XML block accompanied by system instructions declaring that record contents are untrusted user data and must never be followed as instructions.
3. **In-SQL Semester and Cohort Isolation (resourceFilter & canReadResource):**
   Students and CRs are strictly bound to their own cohort_id (or audience_scope = 'all_students') and their current registered semester (cohorts.current_semester). Teachers are restricted to assignedSemesters resolved from teacher_subjects , while Admins have full read access across Semesters 1 - 8.

4. **Post-Generation Output Sanitization and Card Filtering (unsafeOutput & revalidateCards):**
   Scans responses against all sensitive process.env values, strips raw URLs from LLM prose in the Kyana v3 path, and constructs resource cards only for record IDs that were physically returned in the SQL query result set and pass a live permission and sourceVersion SHA-256 check.
5. **Historical Conversation Revalidation (lib/ai-chat-service.js):**
   Whenever historical messages are loaded via getMessages , each assistant turn's stored metadata.accessContext is compared against the user's current contextKey(context) and its evidenceReferences are revalidated against live tables via revalidateCards . If permissions changed or a referenced source was deleted, hidden, or modified, the historical message is replaced with "This earlier reply is no longer available. Search again to check current sources."
6. **Rate Limiting (lib/chat-ratelimit.js):**
   Provides a configurable hourly sliding-window rate limiter (HOURLY_LIMIT = 100 messages per hour keyed by student:${studentId} , session:${sessionID} , or ip:${req.ip}).

7. Web and Mobile Client Architectures

7.1 Web Workspace Client (chatbot.html , chatbot.js , chatbot.css)

The web interface in public/chatbot.html and public/chatbot.js implements a full-page monochrome workspace guarded by SemesterAuth.protectPage() :

- **Streaming SSE Parser (createEventStreamParser & readChatResponse):**
  Reads response.body.getReader() using a streaming TextDecoder , handling arbitrary byte splits across \r\n boundaries, enforcing a 1 MB maximum event size guard, and batching DOM updates via requestAnimationFrame .
- **Code Block Protection, Custom Lexer, and "Run in Compiler" Integration (KyanaCodeBlock):**
  Shields fenced and inline code blocks (___KYANA_PROTECTED_CODE_${counter}___) during LaTeX delimiter normalization, highlights syntax via Prism.js or a zero-dependency single-pass sequential lexer in public/chatbot.js , and injects a Run in Compiler button for supported languages (java , c , cpp , html , css , javascript , python , sql) that bridges directly into public/compiler.html via sessionStorage .
- **Markdown and KaTeX Math Rendering (renderFormattedContent):**
  Parses Markdown via marked (v12.0.2), sanitizes HTML through DOMPurify (v3.1.6), and renders scientific notation via KaTeX (v0.16.11).
- **Interactive Resource Cards (KyanaSourceCard) & Session Sync:**
  Renders clickable cards beneath assistant responses pointing to /api/ai/resources/${card.type}/${card.id}/open and synchronizes local browser sessions with /api/ai/conversations .

7.2 React Native / Expo Mobile Client (kyana.tsx , kyana.ts , kyana-db.ts)

The mobile client in mobile/app/kyana.tsx , mobile/services/kyana.ts , and mobile/services/kyana-db.ts provides a native iOS and Android experience:

- **XHR Incremental SSE Streaming (sendKyanaMessage):**
  Implements SSE streaming over XMLHttpRequest using xhr.onprogress , slicing xhr.responseText.slice(processedIndex) to parse data: lines in real time and invoking onDelta as tokens arrive, with a 60-second timeout and AbortSignal cancellation.
- **Local SQLite WAL Persistence (mobile/services/kyana-db.ts):**
  Uses expo-sqlite (semester_library_chat.db) in Write-Ahead Logging (PRAGMA journal_mode = WAL) mode with a serialized writeQueue transaction lock (withExclusiveTransactionAsync) and an in-memory map (memoryMessages).
- **Native Gestures and Revalidated Card Routing:**
  Renders an inverted FlatList with KyanaSwipeableBubble (swipe-right-to-reply with expo-haptics feedback), PostMarkdown rendering, and native resource cards that call resolveKyanaResource(card) before pushing the target Expo Router screen.

8. Peer-to-Peer Chat Subsystems (Cohort Chat and Direct Messaging)

Alongside Kyana AI, the platform includes two human-to-human messaging systems that are intentionally isolated from AI tables:

1. **Cohort Group Chat (lib/cohort-chat.js & routes/cohort-chat.js):**
   Assigns each BIT intake cohort to one of 8 named group slots (MERCURY , VENUS , EARTH , MARS , JUPITER ,

SATURN , URANUS , NEPTUNE in migrations/002-cohort-chat.js). Enforces transactional room locking (FOR UPDATE), idempotent message sending via chat_send_keys tombstones, @username mentions, emoji reactions, read receipts, 4-second coalesced typing indicators, 75-second online session heartbeats, CR/Admin pinned announcements, and lifecycle operations (semester advancement, graduation, epoch rotation, and slot recycling).

2. **One-to-One Direct Messaging (lib/dm-service.js & routes/direct-messaging.js):**
   Enforces verified-user checks (verification_status = 'verified'), canonical sorted participant pairs, bilateral blocking (dm_blocks), distributed PostgreSQL rate-limiting (dm_rate_limits), single-CTE PostgreSQL message insertion, and background outbox draining (drainDmRealtimeOutbox and drainDmPushOutbox).

9. Evaluation Suites, Benchmarks, and Architectural Findings

The repository includes three automated evaluation harnesses that benchmark the chatbot's conversational quality, hybrid retrieval precision, and local note extraction:

- **150-Scenario Conversational Benchmark (scripts/eval-kyana-conversational.js & tests/data/kyana-evaluation-dataset.json):**
  Tests 14 categories including greetings , slang , spelling_variations , exam_stress , humor_and_transparency , academic_english , academic_romanized_nepali , academic_mixed , academic_devanagari , language_switching , multi_turn_context , library_search , nepali_dates , and semester_security .
- **Hybrid Retrieval Benchmark (scripts/evaluate-kyana-retrieval.js):**
  Evaluates Recall@5 , Precision@5 , MRR (Mean Reciprocal Rank), falsePositiveRate , noAnswerAccuracy , and p95SearchMs across keyword_only and hybrid modes in an isolated disposable PostgreSQL schema.
- **Local Notes Extraction & Search Benchmark (scripts/evaluate-kyana-local-notes.js):**
  Indexes 60 real study note files from the local snapshot into a disposable PostgreSQL schema and measures Recall@5 , MRR , and indexing throughput (resourcesPerSecond).

10. References and Source Code Index

- ai-assistant.js - Core Kyana AI orchestrator, system prompt assembly, v3 knowledge retrieval flow, and tool dispatch.
- server.js - Express application entry point, authentication middleware, /api/ai/* routes, and /api/chat/* mounts.
- lib/chat-http.js - Streaming SSE (text/event-stream) and JSON HTTP transport handler for Kyana AI.
- lib/ai-chat-service.js - Conversation and message persistence (ai_conversations , ai_messages) with historical permission revalidation.
- lib/chat-provider.js - Multi-provider LLM client (Gemini, Groq, OpenRouter, OpenAI, DeepSeek), SSE reader, tool-calling loop, and live web search grounding.
- lib/chat-personality.js - Persona prompt (v2.2.0), 26 few-shot examples, 4-way language detector, and emotional/stress tone adaptation.
- lib/chat-mode.js - Session mode switcher (roast vs. formal) with code-block masking.
- lib/chat-tools.js - Deterministic NLU intent router (routeQuery), subject/semester entity extractor, and tool executor (TOOLS).
- lib/chat-topics.js - Academic topic-to-course mapping dictionary across BIT semesters.
- lib/ai-search-tools.js - Semester-authorized SQL search functions for posts, notices, assignments, and files.
- lib/knowledge-schema.js - Kyana 3.0 feature-flag allowlist (isKnowledgeV3Enabled) and migration manager.
- migrations/20261014_kyana_v3_knowledge.up.sql - PostgreSQL pgvector and weighted FTS schema for knowledge_resources , knowledge_chunks , and knowledge_jobs .
- lib/knowledge-builder.js - Contextual text chunker (600 - 800 chars, 100-char overlap), SHA-256 content hashing, and authority tier assignment.
- lib/knowledge-embeddings.js - gemini-embedding-001 768-dimensional vector embedding client with batching and L2 normalization.
- lib/knowledge-search.js - Single-query hybrid vector + FTS search engine with in-SQL RBAC joins and live hash freshness checks.
- lib/knowledge-relevance.js - Nepali/English stopword filtering, Devanagari/typo alias expansion, named entity gating, and candidate scoring.
- lib/knowledge-sync.js - Real-time lifecycle indexing hooks for posts, comments, assignments, and study notes.
- lib/knowledge-worker.js - Background retry worker (FOR UPDATE SKIP LOCKED) and drift reconciliation engine.
- lib/knowledge-timing.js - Privacy-preserving per-request latency telemetry ring buffer.

- lib/kyana-access.js - Role, cohort, and semester SQL filter generator (resourceFilter) and live card revalidator (revalidateCards).
- lib/kyana-resource-http.js - Revalidated resource redirect and metadata endpoint (/api/ai/resources/:type/:id/open).
- lib/note-search.js - PDF, DOCX, PPTX, TXT, MD, and CSV text extractor and FTS5/GIN note search index.
- lib/university-sync.js - Gandaki University WordPress REST API synchronizer for Tier 1 official notices.
- lib/nepali-dates.js - Nepal Standard Time (UTC+05:45) temporal expression parser.
- lib/chat-security.js - Prompt injection filter, secret leak detector, and contextKey SHA-256 generator.
- lib/chat-ratelimit.js - Hourly sliding-window rate limiter per student, session, or IP.
- routes/internal-knowledge.js - Cron-authenticated worker, reconciliation, university sync, and timing telemetry endpoints.
- public/chatbot.html & public/chatbot.js - Web AI workspace UI, SSE stream consumer, custom syntax highlighter, KaTeX/DOMPurify renderer, and Code Lab compiler bridge.
- mobile/app/kyana.tsx , mobile/services/kyana.ts , & mobile/services/kyana-db.ts - Expo React Native chat screen, XHR SSE client, and local SQLite WAL cache.
- lib/cohort-chat.js & lib/dm-service.js - Peer-to-peer Cohort Group Chat and Direct Messaging services.